

DOCKER COMPOSE EN PRATIQUE

Faire tourner une application de deux services, d'abord à la main puis avec Docker Compose. Le fichier compose.yaml, les commandes utiles, l'attente d'un service prêt, les variables et la mise à l'échelle.

~50

MINUTES

2

SERVICES : DB +
WEB

0

LIGNE DE CODE

7

PARTIES



TP — À la découverte de Docker Compose

Dans ce TP, vous allez faire tourner une application composée de deux services : une base de données PostgreSQL et une interface web d'administration, Adminer. Vous commencerez à la main, conteneur par conteneur, puis vous ferez la même chose avec Docker Compose. Vous n'avez aucun code à écrire : vous utilisez des images officielles.

Prérequis : Docker Engine et le plugin Compose. Vérifiez que la commande `docker compose version` répond avant de commencer.

Durée : environ 50 minutes.

Partie 1 — Lancer une application à la main

Vous allez lancer PostgreSQL puis Adminer, en les plaçant sur un même réseau pour qu'ils communiquent. Exécutez les commandes dans l'ordre :

```
# 1. Créer le réseau et le volume
docker network create demo-net
docker volume create dbdata

# 2. Lancer la base, avec son mot de passe et sa persistance
docker run -d --name db --network demo-net \
  -e POSTGRES_PASSWORD=formation -e POSTGRES_DB=demo \
  -v dbdata:/var/lib/postgresql postgres:18-alpine

# 3. Lancer l'interface web, sur le même réseau
docker run -d --name adminer --network demo-net -p 8080:8080 adminer:4
```

Ouvrez `http://localhost:8080` (ou `http://IP_DE_LA_VM:8080`) dans votre navigateur, puis connectez-vous avec :

CHAMP	VALEUR
Système	PostgreSQL
Serveur	db
Utilisateur	postgres
Mot de passe	formation
Base de données	demo

Vous êtes connecté à la base : l'application fonctionne. Supprimez maintenant tout ce que vous venez de créer, vous allez le refaire autrement :

```
docker rm -f db adminer
docker network rm demo-net
docker volume rm dbdata
```

Partie 2 — La même application avec Docker Compose

Créez un dossier de travail et un fichier `compose.yaml` :

```
mkdir tp-compose
cd tp-compose
nano compose.yaml
```

Saisissez le contenu suivant :

```
services:
  db:
    image: postgres:18-alpine
    environment:
      POSTGRES_PASSWORD: formation
      POSTGRES_DB: demo
    volumes:
      - dbdata:/var/lib/postgresql

  adminer:
    image: adminer:4
    ports:
      - "8080:8080"
    depends_on:
      - db

volumes:
  dbdata:
```

Lancez toute l'application avec une seule commande :

```
docker compose up -d
```

Rouvrez <http://localhost:8080> et reconnectez-vous avec les mêmes informations qu'en Partie 1. Le champ **Serveur** vaut `db` : c'est le nom du service dans le fichier. Compose place les services sur un même réseau et les rend joignables par leur nom, sans adresse IP.

Partie 3 — Comprendre le fichier `compose.yaml`

Voici le rôle de chaque clé du fichier que vous venez d'écrire :

CLÉ	RÔLE
<code>services:</code>	La liste des services de l'application. Chaque service (<code>db</code> , <code>adminer</code>) est joignable par son nom.
<code>image:</code>	L'image utilisée. On peut aussi écrire <code>build: .</code> pour construire une image depuis un Dockerfile local.
<code>environment:</code>	Les variables d'environnement passées au conteneur.
<code>volumes:</code> (dans le service)	Monte un volume nommé pour conserver les données de la base.
<code>ports:</code>	Publie un port vers l'hôte, au format <code>"hôte:conteneur"</code> .
<code>depends_on:</code>	Définit l'ordre de démarrage : <code>adminer</code> démarre après <code>db</code> .
<code>volumes:</code> (au niveau racine)	Déclare les volumes nommés utilisés par les services.

Le fichier ne commence pas par une clé `version:` : elle n'est plus utilisée avec Compose. Le fichier s'appelle `compose.yaml` (l'ancien nom `docker-compose.yml` reste accepté).

Partie 4 — Les commandes utiles

Toutes ces commandes se lancent depuis le dossier qui contient le `compose.yaml` . Essayez-les dans l'ordre :

```
# Voir l'état des services
docker compose ps

# Suivre les journaux de tous les services (Ctrl+C pour quitter)
docker compose logs -f

# Exécuter une commande dans un service, par son nom
docker compose exec db psql -U postgres -d demo -c "\l"

# Vérifier le fichier et afficher la configuration finale
docker compose config

# Arrêter et supprimer les services et le réseau
docker compose down

# Idem, en supprimant aussi les volumes (efface les données de la base)
docker compose down -v
```

Partie 5 — Attendre que la base soit prête

`depends_on` impose l'ordre de démarrage, mais ne vérifie pas que la base est prête à répondre. Pour cela, ajoutez un contrôle de santé (`healthcheck`) sur la base, et une condition sur le service qui en dépend. Modifiez votre `compose.yaml` :

```
services:
  db:
    image: postgres:18-alpine
    environment:
      POSTGRES_PASSWORD: formation
      POSTGRES_DB: demo
    volumes:
      - dbdata:/var/lib/postgresql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d demo"]
      interval: 5s
      timeout: 3s
      retries: 5

  adminer:
    image: adminer:4
    ports:
      - "8080:8080"
    depends_on:
      db:
        condition: service_healthy

volumes:
  dbdata:
```

Relancez, puis vérifiez l'état :

```
docker compose up -d
docker compose ps
```

La base affiche l'état `healthy`, et `adminer` n'a démarré qu'une fois ce contrôle réussi.

Partie 6 — Externaliser la configuration avec un fichier `.env`

Compose lit automatiquement un fichier nommé `.env` placé à côté du `compose.yaml`. Créez-le :

```
nano .env
```

```
POSTGRES_PASSWORD=formation
POSTGRES_DB=demo
```

Puis référez ces variables dans le `compose.yaml`, dans le service `db` :

```
environment:
  POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  POSTGRES_DB: ${POSTGRES_DB}
```

Relancez avec `docker compose up -d`. Le comportement est identique, mais les valeurs ne sont plus écrites en dur dans le fichier. Un fichier `.env` reste en clair : ne l'utilisez pas pour de vrais secrets, et ne le versionnez pas dans Git.

Partie 7 — Lancer plusieurs répliques d'un service

Compose peut lancer plusieurs instances d'un même service. Dans un nouveau dossier, créez un `compose.yaml` avec un petit service web qui affiche le nom du conteneur qui répond :

```
services:
  web:
    image: traefik/whoami
    ports:
      - "8080-8082:80"
```

Lancez trois répliques, puis interrogez-les :

```
docker compose up -d --scale web=3
curl localhost:8080 | grep Hostname
curl localhost:8081 | grep Hostname
docker compose ps
```

Chaque appel répond avec un nom d'hôte différent, et `docker compose ps` liste `web-1`, `web-2` et `web-3`. La plage de ports `"8080-8082:80"` attribue un port par réplique. Avec un port fixe comme `"8080:80"`, la mise à l'échelle échouerait, car un port de l'hôte ne peut être utilisé que par un seul conteneur.

Nettoyage

À la fin du TP, arrêtez et supprimez les stacks depuis chacun de vos dossiers de travail :

```
docker compose down -v
```